UCSE CLI

Manual de uso · Programación 1 (UCSE)

Versión 2.1.0
nuget.org/packages/UcseCLI
github.com/BauDev0/UcseCLI

UCSE CLI arma por vos la estructura de los proyectos de la materia: crea la solución, los proyectos, las carpetas de capas, las referencias entre proyectos y los paquetes, y deja código de ejemplo para que lo adaptes. Lo que normalmente son diez comandos de `dotnet` y varias carpetas a mano, es una línea.

> **Importante:** la herramienta *no resuelve el trabajo práctico*. El código que genera es un ejemplo sobre una entidad de muestra: el modelo del enunciado, las reglas de negocio y los tests de esas reglas los escribís vos.

> **El código de ejemplo viene comentado.** Así podés borrar lo que no uses sin que el resto del proyecto quede lleno de errores. Para verlo funcionando, seleccioná el bloque y descomentalo (`Ctrl+K Ctrl+U` en VS Code y en Visual Studio; `Ctrl+/` alterna), o generá todo activo con `--ejemplos-activos`.

1 Instalación

Requisitos

QUÉ	DETALLE
.NET SDK	Se necesita el **SDK 10** para que corra la herramienta. Los proyectos que genera podés dejarlos en .NET 8, que es la versión de la materia (ver `--framework`).
Git	Opcional. Si lo tenés instalado, la herramienta inicializa el repositorio de cada solución nueva.

```
dotnet --version        # verificar que el SDK esté instalado
```

Si no lo tenés, bajalo de `dotnet.microsoft.com/download`.

Instalar la herramienta

```
# instalar
dotnet tool install --global UcseCLI

# actualizar cuando salga una versión nueva
dotnet tool update --global UcseCLI

# verificar que quedó andando
ucse info
```

`ucse info` te muestra el SDK detectado y la lista de comandos. Lo mismo aparece si escribís `ucse` sin nada. Para las opciones de un comando puntual: `ucse <comando> --help`.

> **Si dice «ucse: command not found»:** cerrá y volvé a abrir la terminal. Las herramientas globales quedan en `~/.dotnet/tools` (Linux y macOS) o `%USERPROFILE%\.dotnet\tools` (Windows), y esa carpeta se agrega al PATH recién en la sesión siguiente.

2 Tu primer proyecto en tres comandos

```
ucse new SistemaVentas     # crea la solución completa
cd SistemaVentas
dotnet build                 # compila de entrada
dotnet run --project SistemaVentas.Consola
```

Con el ejemplo comentado, `dotnet test` todavía no encuentra tests: van a aparecer cuando escribas los tuyos o cuando descomentes los de muestra. Si querés ver todo funcionando desde el primer momento:

```
ucse new SistemaVentas --ejemplos-activos
cd SistemaVentas
dotnet test                 # 5 tests de ejemplo en verde
```

3 Qué genera y dónde va cada cosa

```
SistemaVentas
├── SistemaVentas.sln
├── .gitignore
├── README.md
│
├── SistemaVentas.Consola            PRESENTACIÓN: menús, Console.WriteLine
│   └── Program.cs
│
├── SistemaVentas.LogicaServicio     LÓGICA Y DATOS
│   ├── Entidades/Producto.cs          clases del dominio
│   ├── Servicios/ProductoService.cs   reglas de negocio
│   └── Datos/ProductoRepositorio.cs   leer y guardar el JSON
│
└── SistemaVentas.LogicaTest         TESTS con NUnit
    └── ProductoServiceTest.cs
```

CARPETA	QUÉ VA AHÍ	QUÉ NO VA AHÍ
`Entidades/`	Clases con los datos del dominio y su comportamiento propio.	Lectura de archivos ni `Console`.
`Servicios/`	Las reglas del enunciado. Reciben parámetros, devuelven objetos o lanzan excepciones.	`Console.WriteLine` ni `Console.ReadLine`.
`Datos/`	La única clase que sabe leer y escribir el archivo JSON.	Reglas de negocio.
`*.Consola`	Menús, mensajes al usuario y lectura de lo que escribe.	Validaciones ni cálculos.
`*.LogicaTest`	Un test por comportamiento, con nombre `Metodo_Escenario_ResultadoEsperado`.	Tests de getters y setters.

Esos cuatro archivos de ejemplo (`Producto.cs`, su repositorio, su servicio y sus tests) vienen comentados: están para leerlos, adaptarlos o borrarlos.

> **Por qué importa:** si el servicio no imprime ni lee por consola, la misma lógica sirve después para la Web API de la Unidad 6 sin reescribir nada. Es exactamente lo que pide la práctica de HelpDesk.

4 Crear soluciones

COMANDO	QUÉ ARMA
`ucse new <Nombre>`	Consola + lógica + tests. Para las Unidades 1 a 5.
`ucse new-webapi <Nombre>`	Web API + lógica + tests. Para la Unidad 6.
`ucse add-api`	Suma la Web API a una solución que ya existe.
`ucse new-frontend [carpeta]`	Frontend HTML + CSS + JS que consume la API. Unidad 7.

Opciones

OPCIÓN	PARA QUÉ SIRVE
`-e`, `--entidad <Nombre>`	Nombre de la entidad de ejemplo. Por defecto `Producto`. Conviene usar una del enunciado: `--entidad Ticket`.
`-f`, `--framework net8.0`	Versión de .NET de los proyectos. Si no la ponés, usa la del SDK que tengas instalado.
`--con-api`	En `new`: agrega también el proyecto Web API.
`--con-consola`	En `new-webapi`: agrega también el proyecto de consola.
`--sin-consola`	Solo lógica y tests, sin proyecto de presentación.
`--ejemplos-activos`	Genera el código de ejemplo sin comentar: compila y los tests corren.
`--sin-ejemplos`	Arma la estructura vacía, sin código de ejemplo.
`--sin-git`	No inicializa el repositorio (útil si ya estás dentro de uno).
`--api <url>`	En `new-frontend`: la URL del endpoint que va a consumir.

```
ucse new HelpDesk --entidad Ticket --framework net8.0
```

Lo que suma la Web API

```
HelpDesk.Api
├── Controllers/TicketController.cs   GET, POST, PUT, DELETE
├── DTOs/TicketRequest.cs             lo que recibe + validaciones
├── DTOs/TicketResponse.cs            lo que devuelve
├── DTOs/TicketMapper.cs              conversión DTO ↔ entidad
├── HelpDesk.Api.http                 peticiones listas para probar
└── Program.cs                        controllers + CORS + servicios
```

Los DTOs quedan en el proyecto de la API y el mapeo se hace en el controller: el servicio sigue trabajando solo con la entidad. Para probar los endpoints: `dotnet run --project HelpDesk.Api` y abrí la URL que te muestra la consola, o ejecutá el archivo `.http` desde VS Code con la extensión REST Client.

5 Agregar código a un proyecto

Todos deducen solos el `namespace` según el proyecto y la carpeta donde los generás, y **nunca pisan un archivo que ya existe**. Los cuatro que generan código que depende de otras clases — `add-servicio`, `add-repositorio`, `add-test` y `add-controller` — lo escriben comentado; los esqueletos salen activos. Agregá `--ejemplos-activos` si los querés listos para compilar.

COMANDO	GENERA
`ucse add-entidad <ruta> <Nombres...>`	Clases del dominio con `Id`.
`ucse add-clase <ruta> <Nombres...>`	Clases vacías.
`ucse add-interfaz <ruta> <Nombres...>`	Interfaces (les pone el prefijo `I`).
`ucse add-abstracta <ruta> <Nombres...>`	Clases abstractas.
`ucse add-enum <ruta> <Nombres...>`	Enumeradores.
`ucse add-servicio <ruta> <Entidad>`	Servicio con alta, baja, modificación y búsqueda.
`ucse add-repositorio <ruta> <Entidad>`	Repositorio que persiste en JSON.
`ucse add-test <ruta> <Clase>`	Tests NUnit con patrón Arrange-Act-Assert.
`ucse add-controller <proyecto-api> <Entidad>`	Controller REST con sus DTOs y su mapper.

Podés crear varios de una, separados por espacios:

```
ucse add-entidad HelpDesk.LogicaServicio/Entidades Usuario Comentario
ucse add-enum HelpDesk.LogicaServicio/Entidades Prioridad Estado
```

Atajo: si a `add-test` le pasás un nombre terminado en `Service`, te genera la batería completa de tests de ejemplo (alta, validación que lanza excepción, búsqueda inexistente, baja y verificación de que los datos persistieron), cada uno sobre un archivo temporal para que no se pisen entre sí.

6 Recetas por unidad

Trabajo con objetos y consola

Unidades 1 y 2. Clases, propiedades, colecciones y LINQ, separando presentación de lógica.

```
ucse new Biblioteca --entidad Libro
cd Biblioteca
ucse add-entidad Biblioteca.LogicaServicio/Entidades Autor Prestamo
dotnet run --project Biblioteca.Consola
```

Persistencia en archivos y tests

Unidades 4 y 5. Reglas de negocio, guardado en JSON y tests que cubran lo válido y lo que debe rechazarse.

```
ucse new HelpDesk --entidad Ticket --sin-consola
cd HelpDesk
ucse add-enum HelpDesk.LogicaServicio/Entidades Prioridad Estado
ucse add-test HelpDesk.LogicaTest TicketService --ejemplos-activos
dotnet test
```

Exponer la lógica como Web API

Unidad 6. Sumar la API sobre la solución que ya tenías, sin reescribir nada.

```
cd HelpDesk
ucse add-api
dotnet run --project HelpDesk.Api
```

Para una entidad más, las cuatro capas en cuatro comandos:

```
ucse add-repositorio HelpDesk.LogicaServicio/Datos Usuario
ucse add-servicio HelpDesk.LogicaServicio/Servicios Usuario
ucse add-controller HelpDesk.Api Usuario
ucse add-test HelpDesk.LogicaTest UsuarioService
```

Después registrá el servicio nuevo en `Program.cs`: la herramienta te recuerda la línea exacta al terminar.

Frontend que consume la API

Unidad 7. Pantallas en HTML, CSS y JavaScript sin frameworks.

```
ucse new-frontend Frontend --entidad Ticket
```

Levantá la API, fijate el puerto que muestra la consola y ajustá `API_URL` en `app.js` si no coincide. Después abrí `index.html` (con Live Server de VS Code se recarga solo).

Trabajo práctico integrador

Unidades 1 a 7. Dos soluciones .NET independientes más el frontend, en carpetas separadas dentro del repositorio del equipo.

```
mkdir 2026-prog1-tpfinal-equipo && cd 2026-prog1-tpfinal-equipo
ucse gitignore
ucse new-webapi ApiGestion    --entidad Evento  --sin-git
ucse new-webapi ApiValidacion --entidad Entrada --sin-git
ucse new-frontend Frontend --entidad Evento
```

Con `--sin-git` las soluciones no crean repositorios propios: el repositorio es el del equipo, uno solo en la raíz.

7 Ejemplos de código para copiar

La herramienta trae fragmentos de referencia de la materia, para no salir de la terminal a buscar el apunte a mitad de un ejercicio:

```
ucse ejemplos                          # lista los temas
ucse ejemplos linq                     # lo muestra en pantalla
ucse ejemplos json --guardar Notas/json.cs  # lo guarda en un archivo
```

TEMA	QUÉ MUESTRA	UNIDAD
`herencia`	Clase abstracta, `override` y polimorfismo.	2
`interfaz`	Contrato, implementaciones y uso por constructor.	2
`excepciones`	Cuándo va `ArgumentException` y cuándo `InvalidOperationException`.	2
`linq`	Filtrar, ordenar, proyectar, agregados y `GroupBy`.	2
`nunit`	Arrange-Act-Assert, `Assert.Throws`, `TestCase`, `Within`.	4
`json`	Repositorio con `JsonConvert` y `File`.	5
`controller`	Controller REST con DTOs y códigos de estado.	6
`fetch`	GET, POST y DELETE desde JavaScript y armado del DOM.	7
`git`	Ramas, commits y pull requests.	3

8 Problemas frecuentes

QUÉ VES	QUÉ HACER
`ucse: command not found`	Cerrá y volvé a abrir la terminal. Si sigue, revisá que `~/.dotnet/tools` esté en el PATH.
«No se encontró el SDK de .NET»	Instalá el SDK desde `dotnet.microsoft.com/download` y comprobá con `dotnet --version`.
Al ejecutar `ucse` pide un framework que no tenés	La herramienta corre sobre .NET 10: instalá ese SDK. Tus proyectos pueden seguir en .NET 8 con `--framework net8.0`.
«Ya existe <Nombre> en esta carpeta»	Elegí otro nombre o movete a otra carpeta: la herramienta nunca escribe sobre una solución existente.
«Nombre inválido»	Usá solo letras, números y guiones bajos, empezando con letra: `SistemaVentas`, no `sistema-ventas`.
«Ya existía (no se modificó)»	No es un error: el archivo ya estaba y la herramienta no lo pisó. Borralo o usá otro nombre si querés regenerarlo.
«No se encontró un archivo de solución»	`add-api` se ejecuta parado en la raíz de la solución, donde está el `.sln`.
`dotnet test` no encuentra ningún test	Es lo esperado mientras el ejemplo siga comentado. Descomentá el archivo de tests o escribí los tuyos.
Descomenté el ejemplo de consola y no compila	Borrá la línea `Console.WriteLine` del final del archivo: los `using` tienen que quedar antes de cualquier sentencia.
El frontend no trae datos	Verificá que la API esté corriendo y que `API_URL` en `app.js` tenga el mismo puerto que muestra `dotnet run`.
El JSON no aparece donde esperabas	Se crea junto al ejecutable, en `bin/Debug/net8.0/`. Es lo normal con rutas relativas.

9 Para tener a mano

Comandos de .NET que vas a usar igual

`dotnet build`	Compilar.
`dotnet test`	Correr los tests.
`dotnet run --project X`	Ejecutar un proyecto.

Ayuda de la herramienta

`ucse info`	Entorno y comandos.
`ucse <cmd> --help`	Opciones del comando.
`ucse --version`	Versión instalada.

> **Recordá:** la herramienta te ahorra el armado, no el análisis. Lo que se evalúa —qué entidades existen, qué reglas tiene el negocio y qué casos hay que probar— sigue siendo tuyo. Empezá borrando la entidad de ejemplo que no uses.

UCSE CLI 2.1.0 · `dotnet tool install --global UcseCLI` · Dudas y reporte de errores: github.com/BauDev0/UcseCLI · Licencia MIT